



Universidad Nacional Autónoma de México
Facultad de Ingeniería.
Semestre 2026 - 2



Materia:

Tema selectos de ingeniería en Computación III:
Realidad Virtual y Realidad Aumentada (2958)

Proyecto Final - Manual Técnico:

Escape Room en Realidad Virtual.



Elaborado por:

López González Erick

Grupo:

02

Semestre:

2026-2

Profesor:

Ing. Arturo Pérez De La Cruz

Fecha de entrega:

20 - mayo - 2026

Descripción general del proyecto

El proyecto consiste en una experiencia de realidad virtual tipo escape room, desarrollada en Unity para dispositivos Meta Quest. La aplicación permite al usuario explorar una habitación virtual, interactuar con objetos mediante hand tracking, presionar botones físicos dentro del entorno y resolver tres acertijos para completar una llave virtual.

La experiencia está enfocada en la temática de entretenimiento, ya que propone una dinámica inmersiva basada en exploración, observación, interacción manual y resolución de acertijos. El jugador debe completar tres pruebas dentro de la habitación para llenar una llave visual dividida en tres partes. Al completar la llave, se muestra un mensaje final y la aplicación regresa al menú principal.

Enlace al repositorio de Github con el proyecto:

<https://github.com/ErickLpG/EscapeRoomVR>

Objetivo técnico

Implementar una aplicación de realidad virtual funcional en Unity que integre:

- Seguimiento de manos mediante Meta Quest.
- Interacción con objetos agarrables.
- Botones físicos accionados mediante hand tracking.
- Interfaces en World Space compatibles con VR.
- Sistema de acertijos.
- Sistema de progreso mediante una llave visual.
- Retroalimentación visual y sonora.

Herramientas y tecnologías utilizadas

Elemento		Descripción
Motor de desarrollo		Unity 6000.3.8f1
Plataforma objetivo		Meta Quest
Tipo de experiencia		Realidad Virtual
Sistema de interacción	de	Hand Tracking
Paquetes principales		Meta XR, Interaction SDK, XR Interaction Toolkit / OpenXR
Lenguaje de programación	de	C#
Sistema de UI		Canvas World Space, TextMeshPro, Image UI
Control de escenas		SceneManager
Control de entrada		Unity Input System
Modelado y assets		Objetos 3D de habitación, botones, llave, pósteres y elementos decorativos

Plataforma objetivo

La aplicación está diseñada para ejecutarse en un dispositivo Meta Quest. El proyecto utiliza hand tracking para permitir que el usuario interactúe con los elementos de la habitación sin depender únicamente de los controles físicos.

La aplicación puede probarse mediante:

- Build And Run directo al visor Meta Quest
- Quest Link, siempre que la configuración de OpenXR y Meta XR esté correctamente habilitada.

Planeación

- Cronograma

Actividad	Inicio	Fin	Abril Sem 1		Abril Sem 2		Abril Sem 3		Abril Sem 4	
Diseño del concepto y mecánicas	06 / abril	12 / abril								
Desarrollo de acertijos	13 / abril	19 / abril								
Implementación de interacción	20 / abril	03 / mayo								
Modelado 3D y prototipos	04 / mayo	17 / mayo								
Optimización y pruebas	18 / mayo	24 / mayo								
Documentación y entrega	25 / mayo	27 / mayo								

Actividad	Inicio	Fin	Abril Sem 5		Mayo Sem 1		Mayo Sem 2		Mayo Sem 3	
Diseño del concepto y mecánicas	06 / abril	12 / abril								
Desarrollo de acertijos	13 / abril	19 / abril								
Implementación de interacción	20 / abril	03 / mayo								
Modelado 3D y prototipos	04 / mayo	17 / mayo								
Optimización y pruebas	18 / mayo	24 / mayo								
Documentación y entrega	25 / mayo	27 / mayo								

Actividad	Inicio	Fin	Mayo Sem 4		Mayo Sem 5		Junio Sem 1		Junio Sem 2	
Diseño del concepto y mecánicas	06 / abril	12 / abril								
Desarrollo de acertijos	13 / abril	19 / abril								
Implementación de interacción	20 / abril	03 / mayo								
Modelado 3D y prototipos	04 / mayo	17 / mayo								
Optimización y pruebas	18 / mayo	20 / mayo								
Documentación y entrega	20 / mayo	20 / mayo								

Estructura general del proyecto

La experiencia se divide en dos escenas principales:

Escena	Función
Menulnicio	Habitación inicial donde el usuario puede familiarizarse con objetos interactivos y elegir iniciar o salir
EscenaJuego	Habitación principal del escape room con los tres acertijos

Configuración de realidad virtual

El proyecto utiliza un rig de interacción compatible con Meta Quest. La configuración incluye:

XR Origin
UnityXRInteractionComprehensive
LeftInteractors
RightInteractors
UnityXRHands
HandsManager
UnityXRLeftHandVisual
UnityXRRightHandVisual

El XR Origin se encarga de la posición del jugador y la cámara principal.

El prefab UnityXRInteractionComprehensive permite la interacción con manos, objetos agarrables, botones y elementos físicos.

Sistema de hand tracking

El hand tracking se usa para:

- Visualizar las manos del jugador.
- Agarrar objetos preconfigurados.
- Presionar botones físicos.
- Interactuar con elementos del entorno.

Para objetos agarrables se utilizaron componentes de Meta Interaction SDK, como:

Grabbable

Hand Grab Interactable

Distance Hand Grab Interactable

Grab Free Transformer

Rigidbody

Collider

Botones físicos

Los botones físicos son objetos 3D con una base, una parte móvil y una zona de activación.

La ZonaActivacion tiene un collider configurado como trigger. Cuando una mano entra en la zona, se ejecuta una animación de pulsado, se activa una luz temporal, se reproduce retroalimentación y se invoca un evento.

Script BotonFisicoInteractivo.cs:

```
using System.Collections;
using UnityEngine;
using UnityEngine.Events;

public class BotonFisicoInteractivo : MonoBehaviour
{
    #region Referencias
    [Header("Referencias")]
    public Transform botonVisual;
    public Light luzFeedback;
    #endregion

    #region Configuracion
    [Header("Configuracion")]
    public string mensajeDebug = "Boton fisico pulsado";
    public string tagActivador = "Hand";
    }
```

```
public Vector3 direccionPresionado = new Vector3(0f, -0.03f, 0f);
public float duracionPresionado = 0.08f;
public float tiempoAntesDeRegresar = 0.12f;
public float tiempoEntrePulsaciones = 0.5f;

[Header("Feedback luz")]
public bool apagarLuzAlIniciar = true;
public float tiempoLuzEncendida = 0.25f;

[Header("Eventos")]
public UnityEvent alPulsar;
#endregion

#region Variables privadas
private Vector3 posicionInicialLocal;
private bool enProceso = false;
private bool estadoInicialLuz = false;
#endregion

#region Unity Methods
private void Awake()
{
    if (botonVisual != null)
    {
        posicionInicialLocal = botonVisual.localPosition;
    }

    if (luzFeedback != null)
    {
        estadoInicialLuz = luzFeedback.enabled;

        if (apagarLuzAlIniciar)
        {
            luzFeedback.enabled = false;
        }
    }
}

private void OnTriggerEnter(Collider other)
{
    if (!other.CompareTag(tagActivador))
    {
        return;
    }

    if (enProceso)
    {
        return;
    }
}
```

```
        Debug.Log(mensajeDebug);

        if (alPulsar != null)
        {
            alPulsar.Invoke();
        }

        StartCoroutine(AnimarPulsacion());
    }
#endregion

#region Animacion
private IEnumerator AnimarPulsacion()
{
    enProceso = true;

    if (luzFeedback != null)
    {
        StartCoroutine(ActivarLuzTemporal());
    }

    Vector3 posicionPresionada = posicionInicialLocal +
    direccionPresionado;

    yield return MoverBoton(posicionInicialLocal, posicionPresionada,
    duracionPresionado);
    yield return new WaitForSeconds(tiempoAntesDeRegresar);
    yield return MoverBoton(posicionPresionada, posicionInicialLocal,
    duracionPresionado);

    yield return new WaitForSeconds(tiempoEntrePulsaciones);

    enProceso = false;
}

private IEnumerator MoverBoton(Vector3 desde, Vector3 hasta, float
duracion)
{
    float tiempo = 0f;

    while (tiempo < duracion)
    {
        tiempo += Time.deltaTime;
        float t = tiempo / duracion;

        botonVisual.localPosition = Vector3.Lerp(desde, hasta, t);

        yield return null;
    }

    botonVisual.localPosition = hasta;
}
```



```
    }

    private IEnumerator ActivarLuzTemporal()
    {
        luzFeedback.enabled = true;

        yield return new WaitForSeconds(tiempoLuzEncendida);

        if (apagarLuzAlIniciar)
        {
            luzFeedback.enabled = false;
        }
        else
        {
            luzFeedback.enabled = estadoInicialLuz;
        }
    }
    #endregion
}
```

Funciones principales:

- Detectar contacto con la mano.
- Animar el botón mediante traslación.
- Encender una luz de retroalimentación.
- Ejecutar eventos configurables desde el Inspector mediante UnityEvent.

Este sistema permite reutilizar el mismo botón para diferentes acciones, como introducir números, presionar símbolos, iniciar el juego o salir.

Interfaz de usuario en VR

Las interfaces se implementaron mediante Canvas en modo:

Render Mode: World Space

Esto permite colocar las interfaces como objetos físicos dentro de la habitación, por ejemplo:

- Pantalla de PC.
- Panel de progreso de llave.
- Mensaje de regreso al menú.
- Panel final.

Para paneles que deben aparecer frente al jugador, se utilizó un sistema que posiciona el canvas frente a la cámara y sigue el movimiento de la cabeza de forma controlada.

Script SeguirCamaraVR.cs:

```
using UnityEngine;

public class SeguirCamaraVR : MonoBehaviour
{
    #region Referencias
    [Header("Referencias")]
    public Transform camaraJugador;
    #endregion

    #region Configuracion
    [Header("Configuracion")]
    public float distanciaFrenteCamara = 1.2f;
    public float alturaExtra = -0.1f;
    public float suavizadoPosicion = 10f;
    public float suavizadoRotacion = 10f;
    public bool seguirSoloCuandoEstaActivo = true;
    #endregion

    #region Unity Methods
    private void Start()
    {
        if (camaraJugador == null && Camera.main != null)
        {
            camaraJugador = Camera.main.transform;
        }

        ActualizarPosicionInmediata();
    }

    private void LateUpdate()
    {
        if (seguirSoloCuandoEstaActivo && !gameObject.activeInHierarchy)
        {
            return;
        }

        if (camaraJugador == null)
        {
            return;
        }

        SeguirCamara();
    }
    #endregion

    #region Movimiento
    private void SeguirCamara()
    {
        Vector3 posicionObjetivo = camaraJugador.position +
```

```
camaraJugador.forward * distanciaFrenteCamara;
    posicionObjetivo.y += alturaExtra;

    Quaternion rotacionObjetivo =
Quaternion.LookRotation(transform.position - camaraJugador.position);

    transform.position = Vector3.Lerp(
        transform.position,
        posicionObjetivo,
        Time.deltaTime * suavizadoPosicion
    );

    transform.rotation = Quaternion.Slerp(
        transform.rotation,
        rotacionObjetivo,
        Time.deltaTime * suavizadoRotacion
    );
}

public void ActualizarPosicionInmediata()
{
    if (camaraJugador == null)
    {
        return;
    }

    Vector3 posicionObjetivo = camaraJugador.position +
camaraJugador.forward * distanciaFrenteCamara;
    posicionObjetivo.y += alturaExtra;

    transform.position = posicionObjetivo;
    transform.rotation = Quaternion.LookRotation(transform.position -
camaraJugador.position);
}
#endregion
}
```

El script permite:

- Colocar el panel frente a la cámara.
- Mantener una distancia cómoda.
- Suavizar el seguimiento.
- Reposicionar el panel cuando se activa.

Sistema para volver al menú inicial

Se implementó una función para regresar al menú inicial manteniendo presionado el botón X del control izquierdo durante 5 segundos.

Script ReturnMainScene.cs:

```
using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.SceneManagement;
using TMPPro;

public class ReturnMainScene : MonoBehaviour
{
    #region Input
    [Header("Input")]
    public InputActionReference accionBotonX;
    #endregion

    #region Configuracion
    [Header("Configuracion")]
    public string nombreEscenaInicio = "MenuInicio";
    public float tiempoNecesario = 5f;
    public bool mostrarDebug = true;
    #endregion

    #region Referencias UI opcionales
    [Header("UI Opcional")]
    public GameObject panelVolverInicio;
    public TextMeshProUGUI textoVolverInicio;
    #endregion

    #region Variables privadas
    private float tiempoPresionado = 0f;
    private bool botonPresionado = false;
    private bool cambioEjecutado = false;
    #endregion

    #region Unity Methods
    private void Awake()
    {
        if (panelVolverInicio != null)
        {
            panelVolverInicio.SetActive(false);
        }

        ActualizarTexto(0f);
    }

    private void OnEnable()
    {

```

```
        if (accionBotonX != null)
        {
            accionBotonX.action.Enable();
            accionBotonX.action.started += OnBotonIniciado;
            accionBotonX.action.canceled += OnBotonSoltado;
        }
    }

    private void OnDisable()
    {
        if (accionBotonX != null)
        {
            accionBotonX.action.started -= OnBotonIniciado;
            accionBotonX.action.canceled -= OnBotonSoltado;
            accionBotonX.action.Disable();
        }
    }

    private void Update()
    {
        if (!botonPresionado || cambioEjecutado)
        {
            return;
        }

        tiempoPresionado += Time.deltaTime;

        ActualizarTexto(tiempoPresionado);

        if (mostrarDebug)
        {
            Debug.Log("Manteniendo X: " + tiempoPresionado.ToString("F1") +
" / " + tiempoNecesario);
        }

        if (tiempoPresionado >= tiempoNecesario)
        {
            VolverAlInicio();
        }
    }
#endregion

#region Input Methods
private void OnBotonIniciado(InputAction.CallbackContext context)
{
    botonPresionado = true;
    tiempoPresionado = 0f;

    if (panelVolverInicio != null)
    {
        panelVolverInicio.SetActive(true);
    }
}
```

```
        SeguirCamaraVR panelSeguirCamara =
panelVolverInicio.GetComponent<SeguirCamaraVR>();

        if (panelSeguirCamara != null)
        {
            panelSeguirCamara.ActualizarPosicionInmediata();
        }
    }

    ActualizarTexto(tiempoPresionado);

    if (mostrarDebug)
    {
        Debug.Log("Boton X presionado. Mantener para volver al
inicio.");
    }
}

private void OnBotonSoltado(InputAction.CallbackContext context)
{
    botonPresionado = false;
    tiempoPresionado = 0f;

    if (panelVolverInicio != null)
    {
        panelVolverInicio.SetActive(false);
    }

    ActualizarTexto(tiempoPresionado);

    if (mostrarDebug)
    {
        Debug.Log("Boton X soltado. Se cancelo volver al inicio.");
    }
}
#endregion

#region UI
private void ActualizarTexto(float tiempoActual)
{
    if (textoVolverInicio == null)
    {
        return;
    }

    float tiempoRestante = Mathf.Max(0f, tiempoNecesario -
tiempoActual);

    textoVolverInicio.text =
        "Mantén presionado X para volver al inicio\n" +
```

```
        "Tiempo restante: " + tiempoRestante.ToString("F1") + " s";
    }
#endregion

#region Cambio de escena
private void VolverAlInicio()
{
    cambioEjecutado = true;

    if (mostrarDebug)
    {
        Debug.Log("Volviendo a la escena de inicio.");
    }

    if (string.IsNullOrEmpty(nombreEscenaInicio))
    {
        Debug.LogWarning("No se asigno el nombre de la escena de
inicio.");
        return;
    }

    SceneManager.LoadScene(nombreEscenaInicio);
}
#endregion
}
```

Funcionamiento:

1. El jugador mantiene presionado el botón X.
2. Aparece un panel frente a la cámara.
3. El texto del panel muestra el tiempo restante.
4. Si el jugador mantiene el botón durante el tiempo requerido, se carga la escena del menú inicial.
5. Si suelta el botón antes de tiempo, se cancela la acción.

Binding recomendado:

<XRController>{LeftHand}/primaryButton

Modo silla para la PC

Para facilitar la interacción con los botones de la PC, se implementó un modo especial de silla. Cuando el jugador se acerca a la zona de la silla, el XR Origin se mueve a una posición más cómoda frente al escritorio.

Script ModoSillaPC.cs:

```
using System.Collections;
using UnityEngine;
using UnityEngine.InputSystem;

public class ModoSillaPC : MonoBehaviour
{
    #region Referencias
    [Header("Referencias")]
    public Transform xrOrigin;
    public Transform camaraJugador;
    public Transform puntoSentadoPC;
    public Transform puntoSalidaSilla;

    [Header("Objetos a desactivar durante modo silla")]
    public Behaviour[] componentesLocomocion;
    public Collider[] collidersADesactivar;
    public GameObject[] objetosADesactivar;
    #endregion

    #region Input
    [Header("Input")]
    public InputActionReference accionSalirSilla;
    #endregion

    #region Configuracion
    [Header("Configuracion")]
    public string tagJugador = "Player";
    public float duracionMovimiento = 0.35f;
    public bool entrarAutomaticamente = true;
    public bool guardarPosicionAntesDeSentarse = true;

    [Header("Ajuste VR")]
    public bool alinearCamaraConPunto = true;
    public bool mantenerAlturaActualDelOrigin = false;

    [Header("Proteccion de reentrada")]
    public float tiempoBloqueoEntradaTrasSalir = 1.5f;
    #endregion

    #region Debug
    [Header("Debug")]
    public bool mostrarDebug = true;
    #endregion

    #region UI Opcional
    [Header("UI Opcional")]
    public GameObject panelModoSilla;
    #endregion
}
```



```
#region Variables privadas
private bool enModoSilla = false;
private bool moviendo = false;
private bool entradaBloqueadaTemporalmente = false;

private Vector3 posicionAntesDeSentarse;
private Quaternion rotacionAntesDeSentarse;
#endregion

#region Unity Methods
private void Awake()
{
    Log("Awake ejecutado.");

    if (panelModoSilla != null)
    {
        panelModoSilla.SetActive(false);
        Log("Panel modo silla desactivado en Awake.");
    }

    if (camaraJugador == null && Camera.main != null)
    {
        camaraJugador = Camera.main.transform;
        Log("Camara jugador asignada automaticamente: " +
camaraJugador.name);
    }

    ValidarReferencias();
}

private void OnEnable()
{
    if (accionSalirSilla != null)
    {
        accionSalirSilla.action.Enable();
        accionSalirSilla.action.performed += OnSalirSilla;
        Log("Input salir silla habilitado: " + accionSalirSilla.name);
    }
    else
    {
        LogWarning("No se asigno accionSalirSilla.");
    }
}

private void OnDisable()
{
    if (accionSalirSilla != null)
    {
        accionSalirSilla.action.performed -= OnSalirSilla;
        accionSalirSilla.action.Disable();
    }
}
```

```
        Log("Input salir silla deshabilitado.");
    }
}

private void OnTriggerEnter(Collider other)
{
    Log("OnTriggerEnter detectado con: " + other.name + " | Tag: " +
other.tag);

    if (!entrarAutomaticamente)
    {
        Log("Entrada ignorada: entrarAutomaticamente esta
desactivado.");
        return;
    }

    if (entradaBloqueadaTemporalmente)
    {
        Log("Entrada ignorada: entrada bloqueada temporalmente tras
salir.");
        return;
    }

    if (!other.CompareTag(tagJugador))
    {
        Log("Entrada ignorada: el tag no coincide. Esperado: " +
tagJugador + " | Recibido: " + other.tag);
        return;
    }

    Log("Jugador entro en zona de modo silla PC.");
    EntrarModoSilla();
}

private void OnTriggerExit(Collider other)
{
    Log("OnTriggerExit detectado con: " + other.name + " | Tag: " +
other.tag);
}
#endregion

#region Entrada y salida modo silla
public void EntrarModoSilla()
{
    Log("Solicitud EntrarModoSilla. enModoSilla: " + enModoSilla + " |
moviendo: " + moviendo);

    if (enModoSilla)
    {
        Log("No entra: ya esta en modo silla.");
        return;
    }
}
```

```
    }

    if (moviendo)
    {
        Log("No entra: actualmente se esta moviendo.");
        return;
    }

    if (xrOrigin == null || puntoSentadoPC == null)
    {
        LogWarning("Faltan referencias para entrar al modo silla.");
        return;
    }

    if (guardarPosicionAntesDeSentarse)
    {
        posicionAntesDeSentarse = xrOrigin.position;
        rotacionAntesDeSentarse = xrOrigin.rotation;

        Log("Posicion antes de sentarse guardada: " +
posicionAntesDeSentarse);
        Log("Rotacion antes de sentarse guardada: " +
rotacionAntesDeSentarse.eulerAngles);
    }

    enModoSilla = true;

    ActivarLocomocion(false);

    if (panelModoSilla != null)
    {
        panelModoSilla.SetActive(true);
        Log("Panel modo silla activado.");
    }

    Vector3 destinoOrigin =
CalcularPosicionOriginParaCamara(puntoSentadoPC);
    Quaternion destinoRotacion = puntoSentadoPC.rotation;

    Log("Entrando a modo silla PC.");
    Log("XR Origin posicion inicial: " + xrOrigin.position);
    Log("Camara posicion inicial: " + ObtenerPosicionCamaraTexto());
    Log("Punto sentado posicion: " + puntoSentadoPC.position);
    Log("Destino calculado para XR Origin: " + destinoOrigin);
    Log("Destino rotacion: " + destinoRotacion.eulerAngles);

    StartCoroutine(MoverXRiginAPunto(destinoOrigin, destinoRotacion,
"ENTRAR"));
}

public void SalirModoSilla()
```

```
{
    Log("Solicitud SalirModoSilla. enModoSilla: " + enModoSilla + " |
moviendo: " + moviendo);

    if (!enModoSilla)
    {
        Log("No sale: no esta en modo silla.");
        return;
    }

    if (moviendo)
    {
        Log("No sale: actualmente se esta moviendo.");
        return;
    }

    Vector3 posicionDestino;
    Quaternion rotacionDestino;

    if (puntoSalidaSilla != null)
    {
        posicionDestino =
CalcularPosicionOriginParaCamara(puntoSalidaSilla);
        rotacionDestino = puntoSalidaSilla.rotation;

        Log("Usando puntoSalidaSilla.");
        Log("Punto salida posicion: " + puntoSalidaSilla.position);
    }
    else
    {
        posicionDestino = posicionAntesDeSentarse;
        rotacionDestino = rotacionAntesDeSentarse;

        Log("No hay puntoSalidaSilla. Usando posicion guardada antes de
sentarse.");
    }

    Log("Saliendo de modo silla PC.");
    Log("XR Origin posicion actual: " + xrOrigin.position);
    Log("Camara posicion actual: " + ObtenerPosicionCamaraTexto());
    Log("Destino salida calculado para XR Origin: " + posicionDestino);
    Log("Rotacion salida: " + rotacionDestino.eulerAngles);

    StartCoroutine(SalirModoSillaCoroutine(posicionDestino,
rotacionDestino));
}

private IEnumerator SalirModoSillaCoroutine(Vector3 posicionDestino,
Quaternion rotacionDestino)
{
    yield return MoverXROriginAPunto(posicionDestino, rotacionDestino,
```

```
"SALIR");

    enModoSilla = false;

    if (panelModoSilla != null)
    {
        panelModoSilla.SetActive(false);
        Log("Panel modo silla desactivado.");
    }

    entradaBloqueadaTemporalmente = true;
    Log("Entrada bloqueada temporalmente por " +
    tiempoBloqueoEntradaTrasSalir + " segundos.");

    ActivarLocomocion(true);

    yield return new WaitForSeconds(tiempoBloqueoEntradaTrasSalir);

    entradaBloqueadaTemporalmente = false;
    Log("Entrada desbloqueada. Ya puede volver a entrar a modo
silla.");
}

private void OnSalirSilla(InputAction.CallbackContext context)
{
    Log("Input salir silla recibido. Control: " + context.control.name
+ " | Fase: " + context.phase);
    SalirModoSilla();
}
#endregion

#region Movimiento XR Origin
private IEnumerator MoverXROriginAPunto(Vector3 posicionDestino,
Quaternion rotacionDestino, string motivo)
{
    moviendo = true;

    Vector3 posicionInicial = xrOrigin.position;
    Quaternion rotacionInicial = xrOrigin.rotation;

    Log("Iniciando movimiento " + motivo);
    Log("Desde posicion: " + posicionInicial);
    Log("Hacia posicion: " + posicionDestino);
    Log("Desde rotacion: " + rotacionInicial.eulerAngles);
    Log("Hacia rotacion: " + rotacionDestino.eulerAngles);

    float tiempo = 0f;

    while (tiempo < duracionMovimiento)
    {
        tiempo += Time.deltaTime;
```

```
        float t = tiempo / duracionMovimiento;

        xrOrigin.position = Vector3.Lerp(posicionInicial,
posicionDestino, t);
        xrOrigin.rotation = Quaternion.Slerp(rotacionInicial,
rotacionDestino, t);

        yield return null;
    }

    xrOrigin.position = posicionDestino;
    xrOrigin.rotation = rotacionDestino;

    Log("Movimiento " + motivo + " terminado.");
    Log("XR Origin posicion final: " + xrOrigin.position);
    Log("Camara posicion final: " + ObtenerPosicionCamaraTexto());

    moviendo = false;
}

private Vector3 CalcularPosicionOriginParaCamara(Transform
puntoObjetivoCamara)
{
    if (!alinearCamaraConPunto || camaraJugador == null)
    {
        Log("Calculo posicion: usando posicion directa del punto
objetivo.");
        return puntoObjetivoCamara.position;
    }

    Vector3 diferencia = puntoObjetivoCamara.position -
camaraJugador.position;
    Vector3 posicionCalculada = xrOrigin.position + diferencia;

    if (mantenerAlturaActualDelOrigin)
    {
        posicionCalculada.y = xrOrigin.position.y;
    }

    Log("Calculo posicion para alinear camara.");
    Log("Camara actual: " + camaraJugador.position);
    Log("Punto objetivo camara: " + puntoObjetivoCamara.position);
    Log("Diferencia aplicada al XR Origin: " + diferencia);
    Log("Posicion calculada XR Origin: " + posicionCalculada);

    return posicionCalculada;
}
#endregion

#region Locomocion
private void ActivarLocomocion(bool activar)
```

```
{
    Log("ActivarLocomocion: " + activar);

    if (componentesLocomocion != null)
    {
        for (int i = 0; i < componentesLocomocion.Length; i++)
        {
            if (componentesLocomocion[i] != null)
            {
                componentesLocomocion[i].enabled = activar;
                Log("Componente locomocion [" + i + "] " +
componentesLocomocion[i].name + " -> " + activar);
            }
            else
            {
                Log("Componente locomocion [" + i + "] esta vacio.");
            }
        }
    }

    if (collidersADesactivar != null)
    {
        for (int i = 0; i < collidersADesactivar.Length; i++)
        {
            if (collidersADesactivar[i] != null)
            {
                collidersADesactivar[i].enabled = activar;
                Log("Collider [" + i + "] " +
collidersADesactivar[i].name + " -> " + activar);
            }
            else
            {
                Log("Collider [" + i + "] esta vacio.");
            }
        }
    }

    if (objetosADesactivar != null)
    {
        for (int i = 0; i < objetosADesactivar.Length; i++)
        {
            if (objetosADesactivar[i] != null)
            {
                objetosADesactivar[i].SetActive(activar);
                Log("GameObject [" + i + "] " +
objetosADesactivar[i].name + " -> " + activar);
            }
            else
            {
                Log("GameObject [" + i + "] esta vacio.");
            }
        }
    }
}
```

```
        }
    }
}
#endregion

#region Validacion y debug
private void ValidarReferencias()
{
    Log("Validando referencias.");

    if (xrOrigin == null)
    {
        LogWarning("xrOrigin no asignado.");
    }

    if (camaraJugador == null)
    {
        LogWarning("camaraJugador no asignada.");
    }

    if (puntoSentadoPC == null)
    {
        LogWarning("puntoSentadoPC no asignado.");
    }

    if (accionSalirSilla == null)
    {
        LogWarning("accionSalirSilla no asignada.");
    }

    Collider trigger = GetComponent<Collider>();

    if (trigger == null)
    {
        LogWarning("Este objeto no tiene Collider. OnTriggerEnter no
funcionara.");
    }
    else
    {
        Log("Collider de zona encontrado: " + trigger.GetType().Name +
" | IsTrigger: " + trigger.isTrigger);

        if (!trigger.isTrigger)
        {
            LogWarning("El Collider de la zona no tiene Is Trigger
activado.");
        }
    }
}

private string ObtenerPosicionCamaraTexto()
```



```
{
    if (camaraJugador == null)
    {
        return "Camara no asignada";
    }

    return camaraJugador.position.ToString();
}

private void Log(string mensaje)
{
    if (!mostrarDebug)
    {
        return;
    }

    Debug.Log("[ModoSillaPC] " + mensaje);
}

private void LogWarning(string mensaje)
{
    Debug.LogWarning("[ModoSillaPC] " + mensaje);
}
#endregion
}
```

Funcionamiento:

1. El jugador entra en un trigger cerca de la silla.
2. Se guarda la posición anterior.
3. Se mueve el XR Origin hacia un punto de referencia frente a la PC.
4. Se desactiva temporalmente la locomoción o colliders que estorben.
5. El jugador puede interactuar con la PC cómodamente.
6. Para salir del modo silla, se presiona el botón secundario del control izquierdo.

Binding recomendado:

<XRController>{LeftHand}/secondaryButton

En Meta Quest este botón corresponde normalmente a: Y

Acertijos implementados

Acertijo 1: PC y reloj

El primer acertijo consiste en observar un reloj dentro de la habitación y usar la hora como código para desbloquear la PC.

La clave se compone de: Hora + minutos

Ejemplo: 04:00 -> 0400

El jugador introduce el código usando botones físicos numéricos conectados al script:

Script AcertijoPC_Reloj.cs:

```
using System.Collections;
using TMPro;
using UnityEngine;
using UnityEngine.Events;

public class AcertijoPC_Reloj : MonoBehaviour
{
    #region Referencias UI
    [Header("Referencias UI")]
    public TextMeshProUGUI textoCodigo;
    public TextMeshProUGUI textoEstado;
    #endregion

    #region Configuracion
    [Header("Configuracion")]
    public string codigoCorrecto = "0400";
    public int longitudMaxima = 4;
    public bool bloquearAlResolver = true;
    public float tiempoMostrarError = 1.2f;
    #endregion

    #region Eventos
    [Header("Eventos")]
    public UnityEvent alResolver;
    public UnityEvent alFallar;
    #endregion

    #region Variables privadas
    private string codigoIngresado = "";
    private bool resuelto = false;
    private bool mostrandoError = false;
    #endregion
}
```

```
#region Unity Methods
private void Start()
{
    ReiniciarPantalla();
}
#endregion

#region Metodos para botones
public void AgregarDigito(string digito)
{
    if (NoPuedeInteractuar())
    {
        return;
    }

    if (codigoIngresado.Length >= longitudMaxima)
    {
        return;
    }

    codigoIngresado += digito;
    ActualizarCodigoPantalla();

    if (textoEstado != null)
    {
        textoEstado.text = "Ingresando codigo";
    }

    Debug.Log("Digito ingresado: " + digito);
}

public void BorrarUltimo()
{
    if (NoPuedeInteractuar())
    {
        return;
    }

    if (codigoIngresado.Length <= 0)
    {
        return;
    }

    codigoIngresado = codigoIngresado.Substring(0,
codigoIngresado.Length - 1);
    ActualizarCodigoPantalla();

    Debug.Log("Se borro el ultimo digito");
}

public void LimpiarCodigo()
```

```
{
    if (resuelto && bloquearAlResolver)
    {
        return;
    }

    codigoIngresado = "";
    ActualizarCodigoPantalla();

    if (textoEstado != null)
    {
        textoEstado.text = "Sistema bloqueado";
    }

    Debug.Log("Codigo limpiado");
}

public void ConfirmarCodigo()
{
    if (NoPuedeInteractuar())
    {
        return;
    }

    Debug.Log("Codigo confirmado: " + codigoIngresado);

    if (codigoIngresado.Length < longitudMaxima)
    {
        StartCoroutine(MostrarErrorTemporal("Codigo incompleto"));
        return;
    }

    if (codigoIngresado == codigoCorrecto)
    {
        CodigoCorrecto();
    }
    else
    {
        CodigoIncorrecto();
    }
}

#endregion

#region Logica interna
private void CodigoCorrecto()
{
    resuelto = true;

    if (textoEstado != null)
    {
        textoEstado.text = "Acceso concedido";
    }
}
```

```
    }

    Debug.Log("Acertijo PC resuelto correctamente");

    if (alResolver != null)
    {
        alResolver.Invoke();
    }
}

private voidCodigoIncorrecto()
{
    Debug.Log("Codigo incorrecto");

    if (alFallar != null)
    {
        alFallar.Invoke();
    }

    StartCoroutine(MostrarErrorTemporal("Codigo incorrecto"));
}

private IEnumeratorMostrarErrorTemporal(string mensaje)
{
    mostrandoError = true;

    if (textoEstado != null)
    {
        textoEstado.text = mensaje;
    }

    yield return new WaitForSeconds(tiempoMostrarError);

    codigoIngresado = "";
    ActualizarCodigoPantalla();

    if (!resuelto && textoEstado != null)
    {
        textoEstado.text = "Sistema bloqueado";
    }

    mostrandoError = false;
}

private boolNoPuedeInteractuar()
{
    if (resuelto && bloquearAlResolver)
    {
        return true;
    }
}
```

```
        if (mostrandoError)
        {
            return true;
        }

        return false;
    }

    private void ReiniciarPantalla()
    {
        codigoIngresado = "";
        ActualizarCodigoPantalla();

        if (textoEstado != null)
        {
            textoEstado.text = "Sistema bloqueado";
        }
    }

    private void ActualizarCodigoPantalla()
    {
        if (textoCodigo == null)
        {
            return;
        }

        string textoMostrado = "";

        for (int i = 0; i < longitudMaxima; i++)
        {
            if (i < codigoIngresado.Length)
            {
                textoMostrado += codigoIngresado[i];
            }
            else
            {
                textoMostrado += "_";
            }

            if (i < longitudMaxima - 1)
            {
                textoMostrado += " ";
            }
        }

        textoCodigo.text = textoMostrado;
    }
#endregion
}
```

Funciones principales:

- Agregar dígitos.
- Borrar último dígito.
- Confirmar código.
- Mostrar errores temporales.
- Limpiar código tras fallar.
- Ejecutar evento al resolver.

Al resolver el acertijo, se registra una parte de la llave

Acertijo 2: Secuencia de símbolos

El segundo acertijo consiste en presionar una secuencia correcta de símbolos inspirada en un póster de la habitación.

Secuencia propuesta: Estrella -> Hoja -> Círculo -> Luna

Script AcertijoSecuenciaSimbolos.cs;

```
using TMPro;
using UnityEngine;
using UnityEngine.Events;

public class AcertijoSecuenciaSimbolos : MonoBehaviour
{
    #region Configuracion
    [Header("Configuracion")]
    public string[] secuenciaCorrecta = { "Estrella", "Hoja", "Circulo",
    "Luna" };
    public bool bloquearAlResolver = true;
    #endregion

    #region Referencias UI
    [Header("Referencias UI")]
    public TextMeshProUGUI textoEstado;
    public TextMeshProUGUI textoProgreso;
    #endregion

    #region Eventos
    [Header("Eventos")]
    public UnityEvent alResolver;
    public UnityEvent alFallar;
    #endregion

    #region Variables privadas
    private int indiceActual = 0;
```

```
private bool resuelto = false;
#endregion

#region Unity Methods
private void Start()
{
    ReiniciarSecuencia();
}
#endregion

#region Metodos publicos para botones
public void PresionarSimbolo(string simbolo)
{
    if (resuelto && bloquearAlResolver)
    {
        return;
    }

    Debug.Log("Simbolo presionado: " + simbolo);

    if (secuenciaCorrecta == null || secuenciaCorrecta.Length == 0)
    {
        Debug.LogWarning("No se configuro la secuencia correcta.");
        return;
    }

    if (simbolo == secuenciaCorrecta[indiceActual])
    {
        SimboloCorrecto();
    }
    else
    {
        SimboloIncorrecto();
    }
}

public void ReiniciarSecuencia()
{
    indiceActual = 0;

    if (textoEstado != null)
    {
        textoEstado.text = "Introduce la secuencia";
    }

    ActualizarProgreso();

    Debug.Log("Secuencia reiniciada");
}
#endregion
```



```
#region Logica interna
private void SimboloCorrecto()
{
    indiceActual++;

    if (textoEstado != null)
    {
        textoEstado.text = "Correcto";
    }

    ActualizarProgreso();

    if (indiceActual >= secuenciaCorrecta.Length)
    {
        ResolverAcertijo();
    }
}

private void SimboloIncorrecto()
{
    Debug.Log("Secuencia incorrecta");

    if (textoEstado != null)
    {
        textoEstado.text = "Secuencia incorrecta";
    }

    if (alFallar != null)
    {
        alFallar.Invoke();
    }

    indiceActual = 0;
    ActualizarProgreso();
}

private void ResolverAcertijo()
{
    resuelto = true;

    if (textoEstado != null)
    {
        textoEstado.text = "Compartimento desbloqueado";
    }

    if (textoProgreso != null)
    {
        textoProgreso.text = "Secuencia completa";
    }

    Debug.Log("Acertijo de simbolos resuelto");
}
```

```
        if (alResolver != null)
        {
            alResolver.Invoke();
        }
    }

    private void ActualizarProgreso()
    {
        if (textoProgreso == null)
        {
            return;
        }

        string progreso = "";

        for (int i = 0; i < secuenciaCorrecta.Length; i++)
        {
            if (i < indiceActual)
            {
                progreso += secuenciaCorrecta[i];
            }
            else
            {
                progreso += "_";
            }

            if (i < secuenciaCorrecta.Length - 1)
            {
                progreso += " ";
            }
        }

        textoProgreso.text = progreso;
    }
    #endregion
}
```

Cada botón físico llama al método:

PresionarSimbolo(string simbolo)

El script compara el símbolo presionado con la secuencia correcta. Si el jugador falla, la secuencia se reinicia. Si completa la secuencia, se ejecuta el evento de resolución.

Acertijo 3: Conteo de objetos

El tercer acertijo consiste en contar objetos de tres categorías dentro de la habitación: Animales, Carros y Barcos

El jugador cuenta la cantidad de cada tipo de objeto y utiliza tres botones físicos para aumentar el conteo correspondiente.

Script AcertijoConteoObjetos.cs:

```
using TMPro;
using UnityEngine;
using UnityEngine.Events;

public class AcertijoConteoObjetos : MonoBehaviour
{
    #region Referencias UI
    [Header("Referencias UI")]
    public TextMeshProUGUI textoAnimales;
    public TextMeshProUGUI textoCarros;
    public TextMeshProUGUI textoBarcos;
    public TextMeshProUGUI textoEstado;
    #endregion

    #region Configuracion conteo correcto
    [Header("Conteo correcto")]
    public int animalesCorrectos = 3;
    public int carrosCorrectos = 2;
    public int barcosCorrectos = 4;
    #endregion

    #region Configuracion general
    [Header("Configuracion")]
    public bool resolverAutomaticamente = true;
    public bool bloquearAlResolver = true;
    public int valorMaximoPorCategoria = 9;
    #endregion

    #region Eventos
    [Header("Eventos")]
    public UnityEvent alResolver;
    public UnityEvent alFallar;
    #endregion

    #region Variables privadas
    private int animalesActuales = 0;
    private int carrosActuales = 0;
    private int barcosActuales = 0;
    private bool resuelto = false;
```

```
#endregion

#region Unity Methods
private void Start()
{
    ReiniciarConteo();
}
#endregion

#region Botones de conteo
public void SumarAnimal()
{
    if (NoPuedeInteractuar())
    {
        return;
    }

    animalesActuales++;
    animalesActuales = LimitarValor(animalesActuales);

    Debug.Log("Animales: " + animalesActuales);

    ActualizarPantalla();
    RevisarSolucionAutomatica();
}

public void SumarCarro()
{
    if (NoPuedeInteractuar())
    {
        return;
    }

    carrosActuales++;
    carrosActuales = LimitarValor(carrosActuales);

    Debug.Log("Carros: " + carrosActuales);

    ActualizarPantalla();
    RevisarSolucionAutomatica();
}

public void SumarBarco()
{
    if (NoPuedeInteractuar())
    {
        return;
    }

    barcosActuales++;
    barcosActuales = LimitarValor(barcosActuales);
}
```

```
        Debug.Log("Barcos: " + barcosActuales);

        ActualizarPantalla();
        RevisarSolucionAutomatica();
    }
#endregion

#region Botones extra opcionales
public void ConfirmarConteo()
{
    if (NoPuedeInteractuar())
    {
        return;
    }

    if (ConteoCorrecto())
    {
        ResolverAcertijo();
    }
    else
    {
        ConteoIncorrecto();
    }
}

public void ReiniciarConteo()
{
    if (resuelto && bloquearAlResolver)
    {
        return;
    }

    animalesActuales = 0;
    carrosActuales = 0;
    barcosActuales = 0;

    if (textoEstado != null)
    {
        textoEstado.text = "Cuenta: animales, carros y barcos";
    }

    ActualizarPantalla();

    Debug.Log("Conteo reiniciado");
}
#endregion

#region Logica
private void RevisarSolucionAutomatica()
{

```

```
        if (!resolverAutomaticamente)
        {
            return;
        }

        if (ConteoCorrecto())
        {
            ResolverAcertijo();
        }
    }

    private bool ConteoCorrecto()
    {
        return animalesActuales == animalesCorrectos &&
            carrosActuales == carrosCorrectos &&
            barcosActuales == barcosCorrectos;
    }

    private void ResolverAcertijo()
    {
        resuelto = true;

        if (textoEstado != null)
        {
            textoEstado.text = "Conteo correcto";
        }

        Debug.Log("Acertijo de conteo resuelto");

        if (alResolver != null)
        {
            alResolver.Invoke();
        }
    }

    private void ConteoIncorrecto()
    {
        if (textoEstado != null)
        {
            textoEstado.text = "Conteo incorrecto";
        }

        Debug.Log("Conteo incorrecto");

        if (alFallar != null)
        {
            alFallar.Invoke();
        }

        ReiniciarConteo();
    }
}
```

```
private int LimitarValor(int valor)
{
    if (valor > valorMaximoPorCategoria)
    {
        return 0;
    }

    return valor;
}

private bool NoPuedeInteractuar()
{
    if (resuelto && bloquearAlResolver)
    {
        return true;
    }

    return false;
}

private void ActualizarPantalla()
{
    if (textoAnimales != null)
    {
        textoAnimales.text = "Animales: " + animalesActuales;
    }

    if (textoCarros != null)
    {
        textoCarros.text = "Carros: " + carrosActuales;
    }

    if (textoBarcos != null)
    {
        textoBarcos.text = "Barcos: " + barcosActuales;
    }
}
#endregion
}
```

Cada botón realiza una acción:

Botón Animales -> SumarAnimal()

Botón Carros -> SumarCarro()

Botón Barcos -> SumarBarco()

Cuando el conteo coincide con los valores correctos configurados en el Inspector, el acertijo se resuelve automáticamente.

Sistema de progreso de llave

El sistema de llave controla el avance general del escape room. Cada acertijo completado llena una tercera parte de una llave visual.

Script GestorLlave.cs:

```
using System.Collections;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
using TMPPro;

public class GestorLlave : MonoBehaviour
{
    #region Referencias UI
    [Header("Referencias UI")]
    public GameObject canvasLlave;
    public CanvasGroup canvasGroupLlave;
    public Image imagenLlaveLlena;
    public TextMeshProUGUI textoProgreso;

    [Header("Panel Final")]
    public GameObject panelFinal;
    public CanvasGroup canvasGroupPanelFinal;
    public TextMeshProUGUI textoFinal;
    public TextMeshProUGUI textoConteoMenu;
    #endregion

    #region Audio
    [Header("Audio")]
    public AudioSource audioSource;
    public AudioClip sonidoParteLlave;
    public AudioClip sonidoLlaveCompleta;
    public AudioClip sonidoPanelFinal;
    #endregion

    #region Configuracion
    [Header("Configuracion")]
    public int partesNecesarias = 3;
    public float tiempoMostrarLlave = 2f;
    public float tiempoAntesDeVolverMenu = 5f;
    public string nombreEscenaMenu = "MenuInicio";

    [Header("Animaciones")]
    public float duracionFadeEntrada = 0.35f;
    public float duracionFadeSalida = 0.35f;
    public float duracionLlenadoLlave = 1.2f;
    public float pausaAntesPanelFinal = 0.8f;
    #endregion

    #region Variables privadas
    private int partesObtenidas = 0;
    private bool llaveCompleta = false;
```



```
private bool animando = false;
private float fillActual = 0f;
private Coroutine rutinaActual;
#endregion

#region Unity Methods
private void Awake()
{
    InicializarUI();
}
#endregion

#region Metodos publicos
public void RegistrarParteLlave()
{
    if (llaveCompleta || animando)
    {
        return;
    }

    partesObtenidas++;

    if (partesObtenidas > partesNecesarias)
    {
        partesObtenidas = partesNecesarias;
    }

    Debug.Log("Parte de llave obtenida: " + partesObtenidas + " / " +
partesNecesarias);

    if (rutinaActual != null)
    {
        StopCoroutine(rutinaActual);
    }

    rutinaActual = StartCoroutine(ProcesoParteLlave());
}

public int ObtenerPartesObtenidas()
{
    return partesObtenidas;
}

public bool LlaveEstaCompleta()
{
    return llaveCompleta;
}
#endregion

#region Flujo principal
private IEnumerator ProcesoParteLlave()
{
    animando = true;

    if (canvasLlave != null)
```

```
{
    canvasLlave.SetActive(true);
}

if (panelFinal != null)
{
    panelFinal.SetActive(false);
}

if (canvasGroupPanelFinal != null)
{
    canvasGroupPanelFinal.alpha = 0f;
}

ReproducirSonido(sonidoParteLlave);

yield return FadeCanvas(canvasGroupLlave, 0f, 1f, duracionFadeEntrada);

float fillObjetivo = ObtenerProgresoNormalizado();

yield return LlenarLlave(fillActual, fillObjetivo, duracionLlenadoLlave);

fillActual = fillObjetivo;
ActualizarTextoProgreso();

if (partesObtenidas >= partesNecesarias)
{
    yield return ProcesoLlaveCompleta();
}
else
{
    yield return new WaitForSeconds(tiempoMostrarLlave);

    yield return FadeCanvas(canvasGroupLlave, 1f, 0f, duracionFadeSalida);

    if (canvasLlave != null)
    {
        canvasLlave.SetActive(false);
    }

    animando = false;
}
}

private IEnumerator ProcesoLlaveCompleta()
{
    llaveCompleta = true;

    Debug.Log("Llave completa. Escape room terminado.");

    ReproducirSonido(sonidoLlaveCompleta);

    yield return new WaitForSeconds(pausaAntesPanelFinal);

    if (panelFinal != null)
```

```
{
    panelFinal.SetActive(true);
}

if (textoFinal != null)
{
    textoFinal.text = "Has terminado";
}

ReproducirSonido(sonidoPanelFinal);

yield return FadeCanvas(canvasGroupPanelFinal, 0f, 1f, duracionFadeEntrada);

yield return ConteoRegresivoMenu();

CargarMenu();
}
#endregion

#region UI
private void InicializarUI()
{
    partesObtenidas = 0;
    llaveCompleta = false;
    animando = false;
    fillActual = 0f;

    if (canvasLlave != null)
    {
        canvasLlave.SetActive(false);
    }

    if (canvasGroupLlave != null)
    {
        canvasGroupLlave.alpha = 0f;
    }

    if (panelFinal != null)
    {
        panelFinal.SetActive(false);
    }

    if (canvasGroupPanelFinal != null)
    {
        canvasGroupPanelFinal.alpha = 0f;
    }

    if (imagenLlaveLlena != null)
    {
        imagenLlaveLlena.fillAmount = 0f;
    }

    ActualizarTextoProgreso();

    if (textoFinal != null)
```

```
{
    textoFinal.text = "";
}

if (textoConteoMenu != null)
{
    textoConteoMenu.text = "";
}
}

private void ActualizarTextoProgreso()
{
    if (textoProgreso != null)
    {
        textoProgreso.text = partesObtenidas + " / " + partesNecesarias;
    }
}

private float ObtenerProgresoNormalizado()
{
    if (partesNecesarias <= 0)
    {
        return 0f;
    }

    return (float)partesObtenidas / partesNecesarias;
}

private IEnumerator FadeCanvas(CanvasGroup canvasGroup, float desde, float hasta,
float duracion)
{
    if (canvasGroup == null)
    {
        yield break;
    }

    float tiempo = 0f;
    canvasGroup.alpha = desde;

    while (tiempo < duracion)
    {
        tiempo += Time.deltaTime;
        float t = tiempo / duracion;

        canvasGroup.alpha = Mathf.Lerp(desde, hasta, t);

        yield return null;
    }

    canvasGroup.alpha = hasta;
}

private IEnumerator LlenarLlave(float desde, float hasta, float duracion)
{
    if (imagenLlaveLlena == null)
```

```
{
    yield break;
}

float tiempo = 0f;
imagenLlaveLlena.fillAmount = desde;

while (tiempo < duracion)
{
    tiempo += Time.deltaTime;
    float t = tiempo / duracion;

    imagenLlaveLlena.fillAmount = Mathf.Lerp(desde, hasta, t);

    yield return null;
}

imagenLlaveLlena.fillAmount = hasta;
}

private IEnumerator ConteoRegresivoMenu()
{
    float tiempoRestante = tiempoAntesDeVolverMenu;

    while (tiempoRestante > 0f)
    {
        if (textoConteoMenu != null)
        {
            textoConteoMenu.text = "Volviendo al menú en " +
Mathf.CeilToInt(tiempoRestante) + "...";
        }

        tiempoRestante -= Time.deltaTime;
        yield return null;
    }

    if (textoConteoMenu != null)
    {
        textoConteoMenu.text = "Volviendo al menú...";
    }
}

#endregion

#region Audio
private void ReproducirSonido(AudioClip clip)
{
    if (audioSource == null || clip == null)
    {
        return;
    }

    audioSource.PlayOneShot(clip);
}

#endregion
```

```
#region Escenas
private void CargarMenu()
{
    if (string.IsNullOrEmpty(nombreEscenaMenu))
    {
        Debug.LogWarning("No se asigno el nombre de la escena del menu.");
        return;
    }

    SceneManager.LoadScene(nombreEscenaMenu);
}
#endregion
}
```

Funcionamiento:

1. Cada acertijo llama a: RegistrarParteLlave();
2. El canvas de la llave aparece con fade.
3. La imagen de la llave se llena lentamente usando fillAmount.
4. Se reproduce un sonido de retroalimentación.
5. Si aún no está completa, el canvas desaparece con fade.
6. Si se completan las tres partes, se muestra el panel final.
7. Se muestra una cuenta regresiva.
8. Se regresa al menú inicial.

Scripts principales del proyecto

Script	Función
BotonFisicoInteractivo.cs	Controla botones físicos, animación, luz y eventos
AcertijoPC_Reloj.cs	Valida el código de la PC basado en el reloj
AcertijoSecuenciaSimbolos.cs	Valida la secuencia de símbolos
AcertijoConteoPorBotones.cs	Controla el conteo de animales, carros y barcos
GestorLlave.cs	Controla el progreso de la llave y el final del juego
VolverInicioConBotonX.cs	Permite regresar al menú manteniendo X
PanelSeguirCamaraVR.cs	Posiciona paneles frente a la cámara
ModoSillaPC.cs	Reposiciona al jugador frente a la PC

Flujo completo de la experiencia

1. El jugador inicia en el menú principal.
2. Puede probar objetos y botones en la habitación de menú.
3. Presiona el botón físico de iniciar.
4. Se carga la habitación principal.
5. El jugador explora el cuarto.
6. Resuelve el acertijo de la PC usando la pista del reloj.
7. Resuelve el acertijo de símbolos usando el póster.
8. Resuelve el acertijo de conteo observando animales, carros y barcos.
9. Cada acertijo llena una parte de la llave.
10. Al completar la llave, aparece el mensaje final.
11. Tras la cuenta regresiva, se regresa al menú principal.

Configuración de escenas

Para que el cambio de escenas funcione correctamente, las escenas deben estar agregadas en: File > Build Profiles > Scene List

o: File > Build Settings > Scenes In Build

Escenas requeridas: MenuInicio y EscenaJuego

Los nombres usados en los scripts deben coincidir exactamente con los nombres de las escenas.

Configuración de entrada

El proyecto utiliza Input System para detectar botones de los controles.

Bindings utilizados:

Acción		Binding sugerido	Uso
Volver al inicio		<XRController>{LeftHand}/primaryButton	Mantener X durante 5 segundos
Salir silla	modo silla	<XRController>{LeftHand}/secondaryButton	Salir del modo PC/silla

Para el movimiento básico se utiliza la configuración existente de movimiento.

Requisitos para ejecutar el proyecto

Hardware

- Meta Quest compatible con hand tracking
- PC para desarrollo
- Cable USB-C o conexión para Build And Run
- Espacio físico suficiente para prueba en VR

Software

- Unity 6000.3.8f1
- Meta XR SDK
- OpenXR configurado
- Android Build Support
- TextMeshPro
- Input System

Procedimiento de ejecución

1. Abrir el proyecto en Unity 6000.3.8f1.
2. Verificar que la plataforma objetivo sea Android.
3. Revisar que OpenXR y Meta XR estén habilitados.
4. Conectar el visor Meta Quest.
5. Agregar las escenas en Build Profiles.
6. Ejecutar Build And Run.
7. Probar hand tracking dentro del visor.
8. Verificar interacción con botones y objetos.

9. Resolver los tres acertijos.
10. Confirmar que la llave se complete y regrese al menú.

Limitaciones actuales

La versión actual del proyecto contempla una experiencia breve de un solo cuarto. No incluye:

- Guardado de partida
- Varios niveles
- Inventario avanzado
- Narrativa extendida

El alcance se mantiene limitado para asegurar una implementación estable y funcional dentro del tiempo disponible.

Posibles mejoras futuras

Como trabajo futuro se podrían agregar:

- Más habitaciones.
- Mayor variedad de acertijos.
- Sistema de inventario.
- Narrativa con audio.
- Subtítulos y accesibilidad.
- Sistema de pistas progresivas.
- Más objetos agarrables.
- Mejoras visuales en iluminación y postprocesado.
- Optimización para publicación comercial.
- Modo de dificultad.
- Registro de tiempo de resolución.

Conclusión técnica

El proyecto implementa una experiencia VR funcional de escape room para Meta Quest, utilizando hand tracking, interacción física, UI en World Space, acertijos y progresión visual. La estructura técnica permite reutilizar componentes como botones físicos, scripts de validación y eventos de resolución, facilitando el mantenimiento y futuras expansiones.

Asset recuperado de:

- Free To Use. (s. f.). *Free music for creators*. Recuperado el 20 de mayo de 2026, de <https://freetouse.com/music>
- Kenney. (s. f.). *Home*. Recuperado el 20 de mayo de 2026, de <https://kenney.nl/>

-
- Meta. (2026). *Meta XR All-in-One SDK*. Unity Asset Store. Recuperado el 20 de mayo de 2026, de <https://assetstore.unity.com/packages/tools/integration/meta-xr-all-in-one-sdk-269657>